

Counting Sort

COUNTING-SORT(A, B, n, k)

for $i \leftarrow 0$ to k

do $C[i] \leftarrow 0$

for $j \leftarrow 1$ to n

do $C[A[j]] \leftarrow C[A[j]] + 1$

for $i \leftarrow 1$ to k

do $C[i] \leftarrow C[i] + C[i - 1]$

for $j \leftarrow n$ downto 1

do $B[C[A[j]]] \leftarrow A[j]$

$C[A[j]] \leftarrow C[A[j]] - 1$

1	2	3	4	5	6	7	8
3	6	4	1	3	4	1	4

1	2	3	4	5	6
2	0	2	3	0	1

1	2	3	4	5	6	7	8
	1					4	

1	2	3	4	5	6
1	2	4	6	7	8

1	2	3	4	5	6	7	8
						4	

1	2	3	4	5	6
2	2	4	6	7	8

1	2	3	4	5	6	7	8
1	1	3	3	4	4	4	6

Do an example for $A = 2_1, 5_1, 3_1, 0_1, 2_2, 3_2, 0_2, 3_3$

Counting sort is *stable* (keys with same value appear in same order in output as they did in input) because of how the last loop works.

Analysis: $\Theta(n + k)$, which is $\Theta(n)$ if $k = O(n)$.

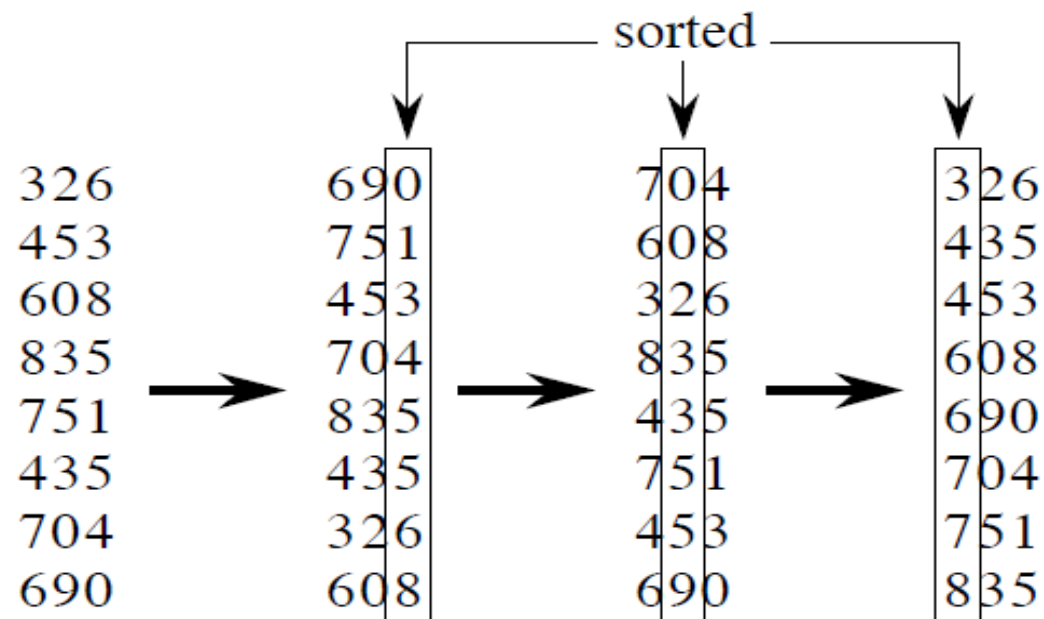
Radix Sort

RADIX-SORT(A, d)

for $i \leftarrow 1$ **to** d

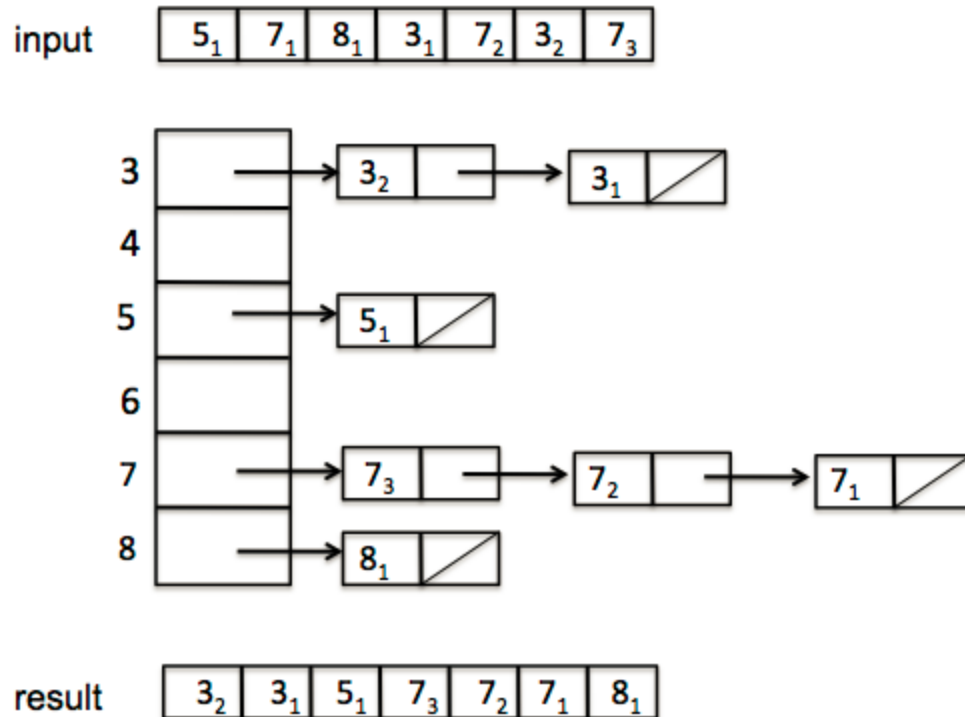
do use a stable sort to sort array A on digit i

Example:

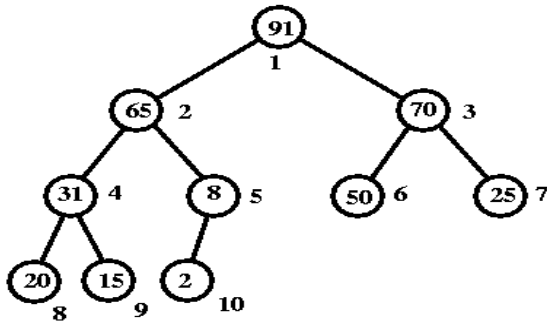


Bucket Sort (Stable sorting)

Bucket Sort



Heap (Priority Q, Implementation, max, min)



1	2	3	4	5	6	7	8	9	10
91	65	70	31	8	50	25	20	15	2

PARENT(i)

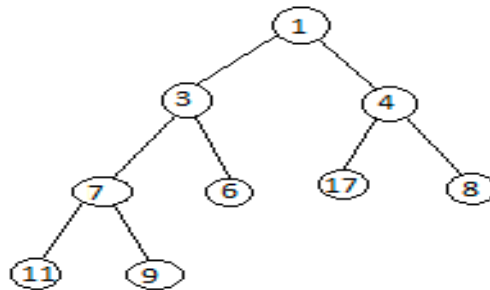
1 return $\lfloor i/2 \rfloor$

LEFT(i)

1 return $2i$

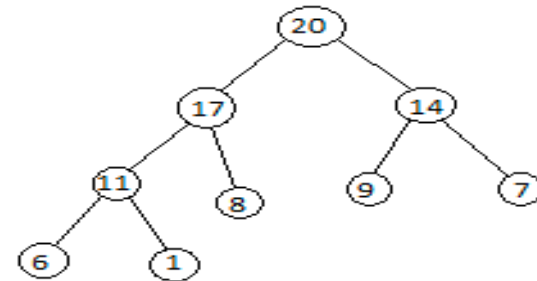
RIGHT(i)

1 return $2i + 1$



Min-Heap

In min-heap, first element is the smallest. So when we want to sort a list in ascending order, we create a Min-heap from that list, and pick the first element, as it is the smallest, then we repeat the process with remaining elements.



Max-Heap

In max-heap, the first element is the largest, hence it is used when we need to sort a list in descending order.

Heapify

MAX-HEAPIFY(A, i)

```
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )
```

Heap Building

```
BUILD-MAX-HEAP'(A)
1 heap-size[A] = 1
2 for  $i = 2$  to length[A]
3   MAX-HEAP-INSERT(A, A[i])
```

Knuth Morris Pratt Pattern Matching.

KMP-MATCHER(T, P)

```
1   $n = T.length$ 
2   $m = P.length$ 
3   $\pi = \text{COMPUTE-PREFIX-FUNCTION}(P)$ 
4   $q = 0$  // number of characters matched
5  for  $i = 1$  to  $n$  // scan the text from left to right
6      while  $q > 0$  and  $P[q + 1] \neq T[i]$ 
7           $q = \pi[q]$  // next character does not match
8      if  $P[q + 1] == T[i]$ 
9           $q = q + 1$  // next character matches
10     if  $q == m$  // is all of  $P$  matched?
11         print "Pattern occurs with shift"  $i - m$ 
12          $q = \pi[q]$  // look for the next match
```

Knuth Morris Pratt Pattern Matching contd..

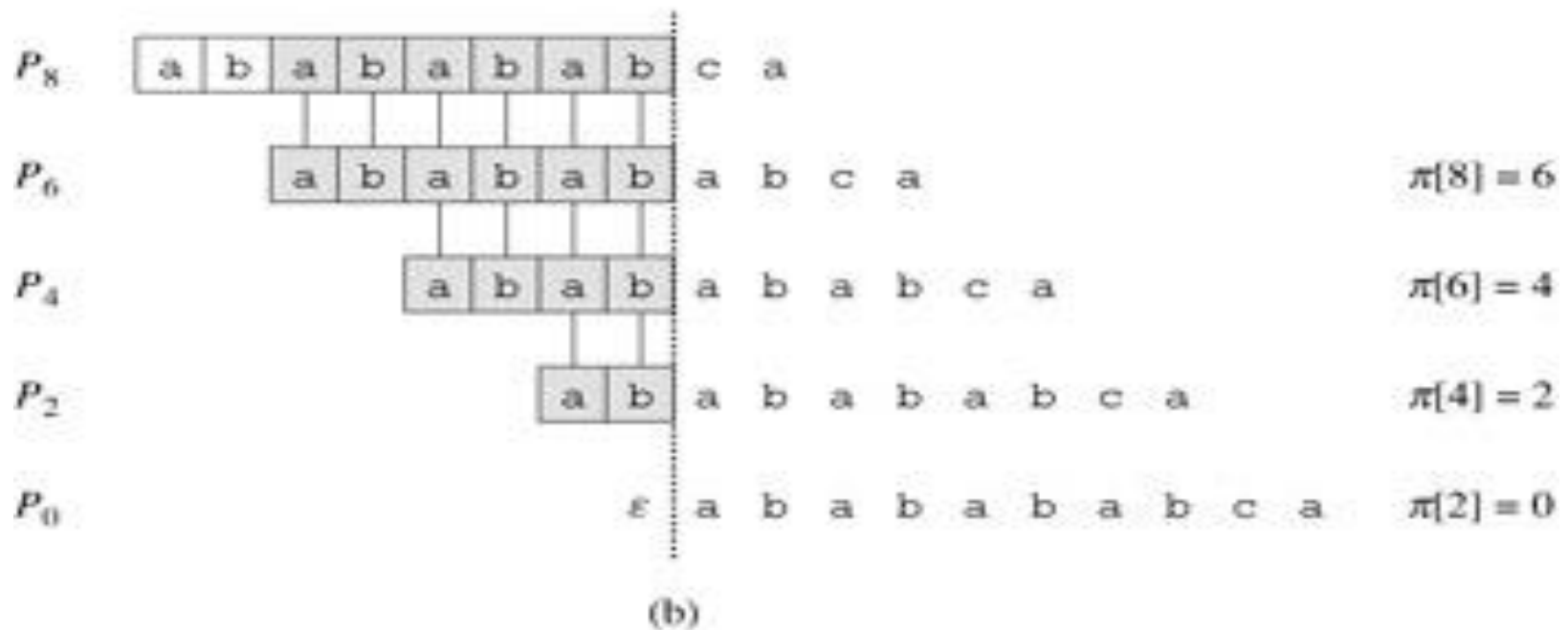
COMPUTE-PREFIX-FUNCTION(P)

```
1   $m = P.length$ 
2  let  $\pi[1..m]$  be a new array
3   $\pi[1] = 0$ 
4   $k = 0$ 
5  for  $q = 2$  to  $m$ 
6      while  $k > 0$  and  $P[k + 1] \neq P[q]$ 
7           $k = \pi[k]$ 
8      if  $P[k + 1] == P[q]$ 
9           $k = k + 1$ 
10      $\pi[q] = k$ 
11 return  $\pi$ 
```

Knuth Morris Pratt: (Comp. Prefix)example

i	1	2	3	4	5	6	7	8	9	10
$P[i]$	a	b	a	b	a	b	a	b	c	a
$\pi[i]$	0	0	1	2	3	4	5	6	0	1

(a)



Median Finding.. Basic idea

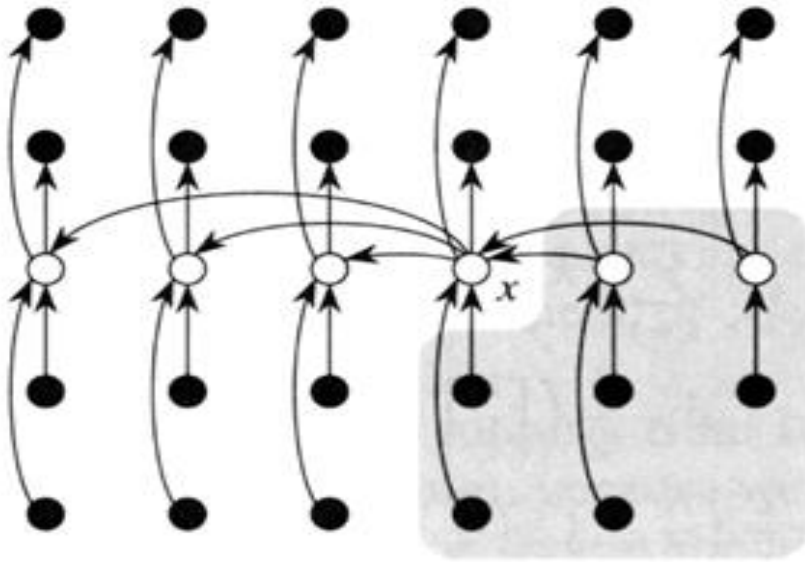


Figure 10.1 Analysis of the algorithm SELECT. The n elements are represented by small circles, and each group occupies a column. The medians of the groups are whitened, and the median-of-medians x is labeled. Arrows are drawn from larger elements to smaller, from which it can be seen that 3 out of every group of 5 elements to the right of x are greater than x , and 3 out of every group of 5 elements to the left of x are less than x . The elements greater than x are shown on a shaded background.

Median finding contd..

1. Divide the n elements of the input array into $\lfloor n/5 \rfloor$ groups of 5 elements each and at most one group made up of the remaining $n \bmod 5$ elements.
2. Find the median of each of the $\lfloor n/5 \rfloor$ groups by first insertion-sorting the elements of each group (of which there are at most 5) and then picking the median from the sorted list of group elements.
3. Use SELECT recursively to find the median x of the $\lfloor n/5 \rfloor$ medians found in step 2. (If there are an even number of medians, then by our convention, x is the lower median.)
4. Partition the input array around the median-of-medians x using the modified version of PARTITION. Let k be one more than the number of elements on the low side of the partition, so that x is the k th smallest element and there are $n - k$ elements on the high side of the partition.
5. If $i = k$, then return x . Otherwise, use SELECT recursively to find the i th smallest element on the low side if $i < k$, or the $(i - k)$ th smallest element on the high side if $i > k$.